```
RUN java -Djarmode=layertools -jar catalog-service.jar extract

FROM eclipse-temurin:17
RUN useradd spring          ❶
USER spring                 ❷
WORKDIR workspace
COPY --from=builder workspace/dependencies/ ./
COPY --from=builder workspace/spring-boot-loader/ ./
COPY --from=builder workspace/snapshot-dependencies/ ./
COPY --from=builder workspace/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.JarLauncher"]
```

❶ Creates a “spring” user

❷ Configures “spring” as the current user

As previously noted, you should never store secrets like passwords or keys in a container image. Even if they are removed in an upper layer, they will remain intact in the original layer and be easily accessible.

Finally, it’s also critical to use up-to-date base images and libraries in your Dockerfile. Scanning your container images for vulnerabilities is a best practice that should be embraced and automated in your deployment pipeline. In chapter 3, you learned how to use grype to scan codebases for vulnerabilities. Now we’ll also use it to scan a container image.

Using the updated Dockerfile, build a new container image for Catalog Service. Open a Terminal window, navigate to the Catalog Service root folder, and run this command (don’t forget the final dot):

```
$ docker build -t catalog-service .
```

Next, use `grype` to check if the newly created image contains any vulnerabilities:

```
$ grype catalog-service
```